

Problem Statement:

Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

Write a menu-driven C program using `switch-case` to input a positive integer and perform one of the following operations based on the user's choice:

- **Check whether all its digits are unique.**

- E.g. 1: Number: 12345 , Output: Unique
- E.g. 2: Number: 12234 , Output: Not Unique

- **Find the frequency of each digit from '0' to '9'.**

- E.g. Number: 112333 , Output: `freq_0 = 0, freq_1 = 2, freq_2 = 1, freq_3 = 3, freq_4 = 0, freq_5 = 0, freq_6 = 0, freq_7 = 0, freq_8 = 0, freq_9 = 0`

1. ALGORITHMS

A. Check Unique Digits

1. Start.
2. Input a positive integer `num` .
3. Initialize frequency array `freq[10] = {0}` .
4. Copy `num` to temporary variable `temp` .
5. While `temp > 0` , extract last digit `digit = temp % 10` , increment `freq[digit]` , and update `temp = temp / 10` .
6. Check if any `freq[i] > 1` for `i` from `0` to `9` . If true, set flag `isUnique = 0` .
7. If `isUnique == 1` , print "Unique" , else print "Not Unique" .
8. Stop.

B. Digit Frequency Counter

1. Start.
2. Input a positive integer `num` .
3. Initialize frequency array `freq[10] = {0}` .
4. Copy `num` to temporary variable `temp` .
5. If `temp == 0` , set `freq[0] = 1` .
6. While `temp > 0` , extract digit `digit = temp % 10` , increment `freq[digit]` , and set `temp = temp / 10` .
7. Print frequency for each digit `0` through `9` in format `freq_i = count` .
8. Stop.

2. TEST CASE TABLE

Test Case ID	Choice	Input Number	Expected Output	Status
TC_01	1	12345	Unique	Pass
TC_02	1	12234	Not Unique	Pass
TC_03	2	112333	freq_0 = 0, freq_1 = 2, freq_2 = 1, freq_3 = 3, freq_4 = 0, freq_5 = 0, freq_6 = 0, freq_7 = 0, freq_8 = 0, freq_9 = 0	Pass
TC_04	1	7	Unique	Pass
TC_05	2	9050	freq_0 = 2, freq_1 = 0, freq_2 = 0, freq_3 = 0, freq_4 = 0, freq_5 = 1, freq_6 = 0, freq_7 = 0, freq_8 = 0, freq_9 = 1	Pass

3. C PROGRAM SOURCE CODE

```
#include <stdio.h>

// Function to check if all digits in a number are unique
void checkUniqueDigits(long long num) {
    int freq[10] = {0};
    long long temp = num;
    int isUnique = 1;

    if (temp == 0) {
        freq[0]++;
    }

    while (temp > 0) {
        int digit = temp % 10;
        freq[digit]++;
        temp /= 10;
    }

    for (int i = 0; i < 10; i++) {
        if (freq[i] > 1) {
            isUnique = 0;
            break;
        }
    }

    if (isUnique) {
        printf("Output: Unique
");
    } else {
        printf("Output: Not Unique
");
    }
}
```

```

// Function to count frequency of each digit (0 to 9)
void findDigitFrequency(long long num) {
    int freq[10] = {0};
    long long temp = num;

    if (temp == 0) {
        freq[0]++;
    }

    while (temp > 0) {
        int digit = temp % 10;
        freq[digit]++;
        temp /= 10;
    }

    for (int i = 0; i < 10; i++) {
        printf("freq_%d = %d", i, freq[i]);
        if (i < 9) {
            printf(", ");
        }
    }
    printf("\n");
}

int main() {
    int choice;
    long long num;

    printf("----- MENU -----");
    printf("\n");
    printf("1. Check whether all digits are unique");
    printf("\n");
    printf("2. Find frequency of each digit (0-9)");
    printf("\n");
    printf("-----");
    printf("\n");
    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter a positive integer: ");
            scanf("%lld", &num);
            if (num < 0) {
                printf("Invalid Input! Number must be positive.");
            }
            else {
                checkUniqueDigits(num);
            }
            break;

        case 2:

```

```
        printf("Enter a positive integer: ");
        scanf("%lld", &num);
        if (num < 0) {
            printf("Invalid Input! Number must be positive.
");
        } else {
            findDigitFrequency(num);
        }
        break;

    default:
        printf("Invalid Choice! Please enter 1 or 2.
");
    }

    return 0;
}
```